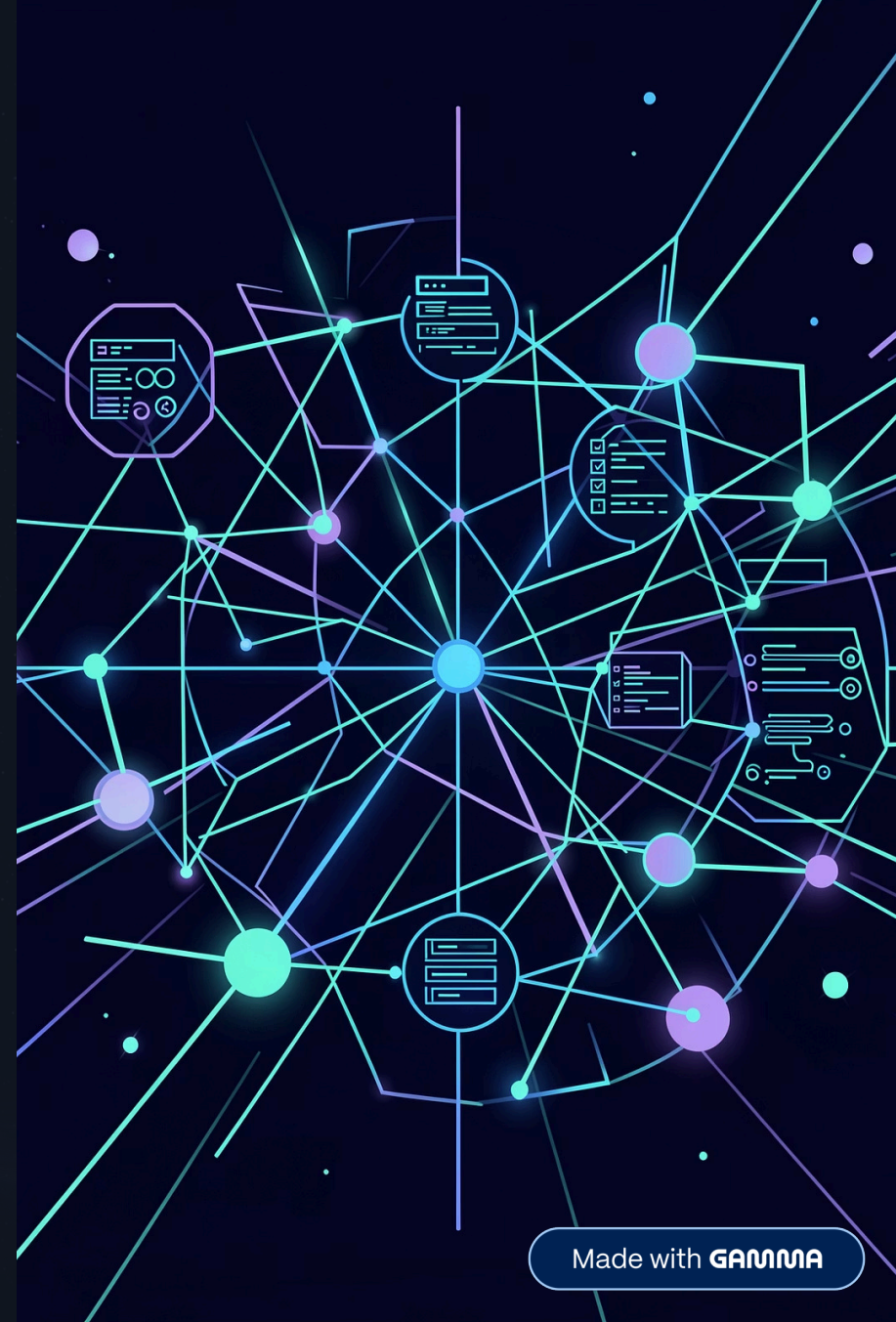


Listas Enlazadas: Creación y Manipulación en Java

DESARROLLO DE SOFTWARE

ESTRUCTURAS DE DATOS

Una exploración técnica de las estructuras dinámicas fundamentales en programación



Made with GAMMA

Concepto de Lista Enlazada

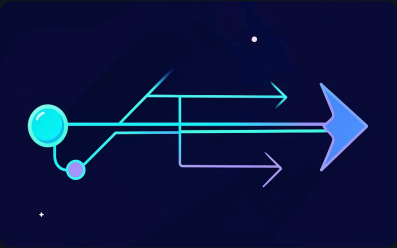


Definición Técnica

Una lista enlazada es una estructura de datos dinámica compuesta por nodos secuenciales, donde cada nodo contiene un dato y una referencia al siguiente elemento.

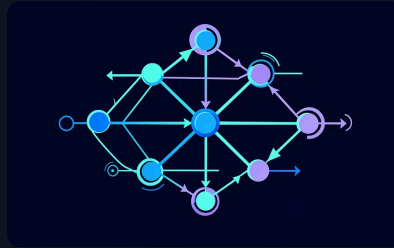
- Almacenamiento no contiguo en memoria
- Tamaño dinámico y flexible
- Acceso secuencial mediante referencias
- Mayor eficiencia en inserciones y eliminaciones que los arrays

Tipos de Listas Enlazadas



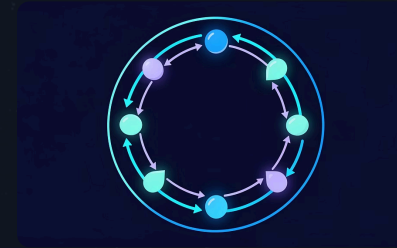
Lista Simplemente Enlazada

Cada nodo contiene una referencia únicamente al nodo siguiente. Recorrido unidireccional de inicio a fin.



Lista Doblemente Enlazada

Cada nodo mantiene referencias tanto al nodo anterior como al siguiente, permitiendo navegación bidireccional.



Lista Circular

El último nodo apunta de vuelta al primero, formando un ciclo continuo sin nodo terminal nulo.

Implementación en Java: Estructura Base

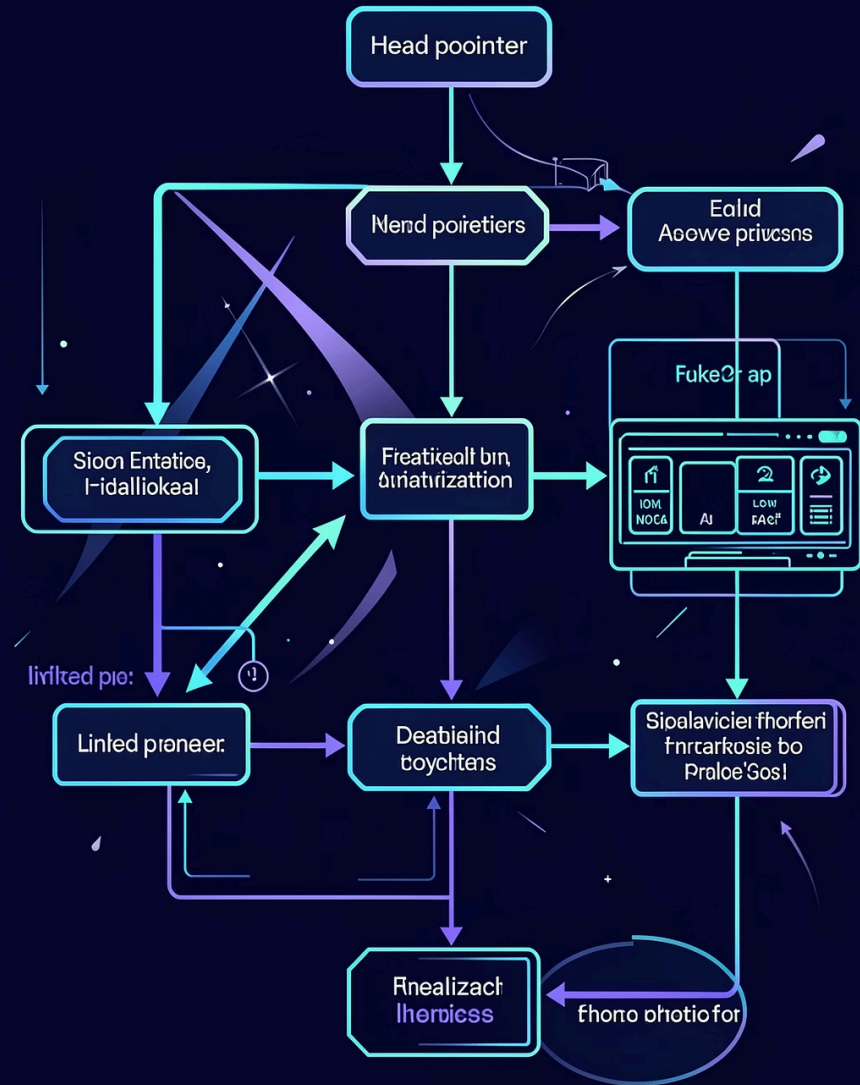
Clase Nodo

El nodo es la unidad fundamental de una lista enlazada. Encapsula tanto el dato almacenado como la referencia al siguiente elemento de la secuencia.

- Atributo `dato`: almacena la información
- Atributo `siguiente`: referencia al próximo nodo
- Constructor para inicializar valores
- Métodos `getter` y `setter` para acceso controlado

```
public class Nodo {  
    private int dato;  
    private Nodo siguiente;  
  
    public Nodo(int dato) {  
        this.dato = dato;  
        this.siguiente = null;  
    }  
  
    public int getDato() {  
        return dato;  
    }  
  
    public Nodo getSiguiente() {  
        return siguiente;  
    }  
  
    public void setSiguiente(Nodo sig) {  
        this.siguiente = sig;  
    }  
}
```

Intuallization Filcad Proesso: ILiNkEL 7et orout a LiL Clorr List



Creación de una Lista Enlazada

01

Declaración de la clase ListaEnlazada

Se crea la clase contenedora con un atributo `head` (cabeza) que apunta al primer nodo. Inicialmente es `null` para indicar lista vacía.

02

Inicialización del primer nodo

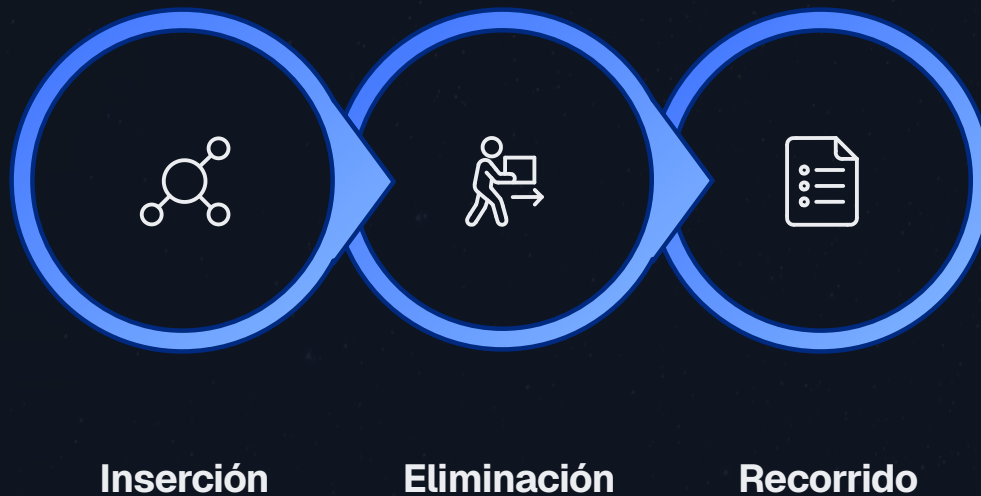
Se instancia un nuevo objeto `Nodo` con el dato inicial. La referencia `head` se actualiza para apuntar a este primer elemento de la estructura.

03

Gestión del puntero head

El atributo `head` siempre mantiene la referencia al inicio de la lista, permitiendo el acceso y recorrido completo de todos los nodos enlazados.

Manipulación de la Lista: Operaciones Fundamentales



Las operaciones de manipulación modifican la estructura dinámica de la lista mediante la actualización estratégica de referencias entre nodos.

Inserción de Nodos

Añadir elementos al inicio, final o posición específica ajustando las referencias del nodo anterior y posterior.

Eliminación de Nodos

Remover elementos redirigiendo la referencia del nodo anterior para saltar el nodo a eliminar.

Recorrido de la Lista

Navegar secuencialmente desde **head** hasta encontrar **null**, procesando cada nodo.

Ejemplo Básico en Java: Construcción Paso a Paso

```
public class ListaEnlazada {
    private Nodo head;

    public ListaEnlazada() {
        this.head = null;
    }

    // Insertar al inicio
    public void insertarInicio(int dato) {
        Nodo nuevoNodo = new Nodo(dato);
        nuevoNodo.setSiguiendo(head);
        head = nuevoNodo;
    }

    // Recorrer e imprimir
    public void recorrer() {
        Nodo actual = head;
        while (actual != null) {
            System.out.print(actual.getDato() + " -> ");
            actual = actual.getSiguiendo();
        }
        System.out.println("null");
    }

    public static void main(String[] args) {
        ListaEnlazada lista = new ListaEnlazada();
        lista.insertarInicio(10);
        lista.insertarInicio(20);
        lista.insertarInicio(30);
        lista.recorrer(); // Salida: 30 -> 20 -> 10 -> null
    }
}
```

Ventajas y Eficiencia de Memoria

Uso Eficiente de Recursos

Las listas enlazadas optimizan el uso de memoria mediante asignación dinámica, creciendo o reduciéndose según las necesidades reales de la aplicación.

Casos de Uso Comunes

- Implementación de pilas y colas
- Gestión de historial de navegación
- Listas de reproducción multimedia
- Sistemas de deshacer/rehacer
- Tablas hash con encadenamiento
- Manejo de procesos en sistemas operativos

Asignación Dinámica

No requiere reserva previa de memoria. Los nodos se crean conforme se necesitan.

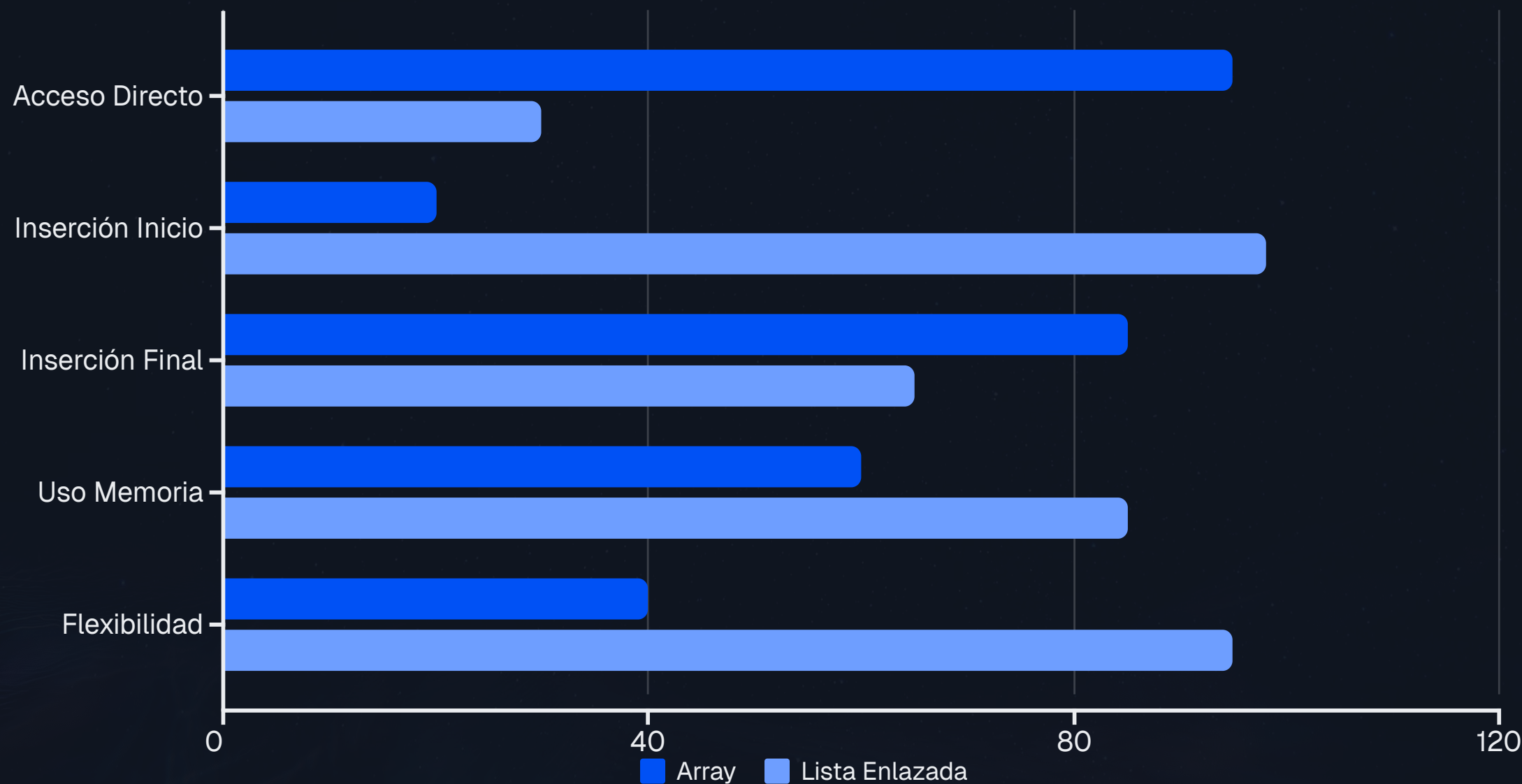
Inserciones $O(1)$

Añadir elementos al inicio es instantáneo, sin desplazamiento de datos.

Sin Fragmentación

Los nodos pueden ubicarse en cualquier posición de memoria disponible.

Comparativa: Arrays vs Listas Enlazadas



Conclusiones y Buenas Prácticas



Validar Referencias Nulas

Siempre verificar que los punteros no sean `null` antes de acceder a métodos para evitar `NullPointerException`.



Liberar Memoria

Establecer referencias a `null` cuando se eliminan nodos para facilitar la recolección de basura en Java.



Encapsulación

Mantener el atributo `head` privado y proporcionar métodos públicos para manipular la lista de forma controlada.



Complejidad Temporal

Elegir la estructura adecuada según operaciones frecuentes: listas para inserciones, arrays para acceso aleatorio.